CLIENTOPS CRM

ClientOps

Multi-tenant CRM SaaS — Laravel 12 + React 19

Przewodnik Implementacji

Krok po kroku jak wdrażać poszczególne moduły systemu — migracje, modele, kontrolery, requesty, testy. Strategia testowania i częstotliwość.

WERSJA	DATA	FAZY	SEKCJI
1.0.0	Marzec 2026	6	9

Spis treści

Dokument zawiera 9 sekcji

1 Wprowadzenie — jak korzystać z tego dokumentu

2 Środowisko deweloperskie

3 Faza 1 — Auth, role, multi-tenancy

4 Faza 2 — CRM: klienci, leady, Kanban, oferty

5 Faza 3 — Tickety i SLA

6 Faza 4 — Dokumenty i PDF

7 Faza 5 — Powiadomienia i wiadomości wewnętrzne

8 Faza 6 — SEO z AI

9 Strategia testowania

Wprowadzenie — jak korzystać z tego dokumentu

Ten dokument to przewodnik dla developera implementującego ClientOps od zera. Każda sekcja odpowiada jednej fazie wdrożenia — od fundamentów auth i multi-tenancy, przez CRM, tickety, dokumenty i powiadomienia, aż po SEO z AI. Podzielone jest to celowo: każda faza jest deployowalna osobno i działa bez kolejnej.

Konwencje w tym dokumencie

ELEMENT	ZNACZENIE
`kod inline`	Nazwy plików, klas, metod, kolumn bazy danych
Bloki kodu	Przykładowe implementacje — nie kopiuj ślepo, dostosuj do kontekstu
✓ Unit	Typ testu rekomendowany dla danej funkcjonalności
Horizon	Operacja asynchroniczna — wymaga działającego workera kolejki
Breaking	Zmiana, która wymaga migracji danych lub zmiany kontraktów API

Plan faz — przegląd

FAZA 1

Fundament

→ Auth (hasło, Google, 2FA)
→ Role (Spatie)
→ Multi-tenancy
→ Panel admina / klienta

FAZA 2

CRM

→ Klienci i kontakty
→ Leady + lead_activities
→ Kanban 7 kolumn
→ Oferty + konwersja

FAZA 3

Tickety + SLA

→ Tickety + komentarze
→ SLA policies
→ SLA incidents (job)
→ Kanban ticketów

FAZA 4

Dokumenty

→ TipTap editor
→ Wersjonowanie
→ PDF (Browsershot)
→ Spatie Media Library

FAZA 5

Powiadomienia

→ Email → in-app
→ SMS (SMSAPI)
→ Reverb WebSocket
→ Wiadomości wewnętrzne

FAZA 6

SEO z AI

→ GSC + GA4 API
→ Crawler + PageSpeed
→ AI (Claude/OpenAI)
→ TimescaleDB pipeline

Zasada: każda faza jest niezależna

System po Fazie 1 działa i można go wdrożyć. Faza 2 rozszerza go o CRM, ale nie wymaga Fazy 3 ani system.

Środowisko deweloperskie

Wymagania

TECHNOLOGIA	WERSJA	UWAGI
PHP	8.3+	Rozszerzenia: pdo_pgsql, redis, gd, zip, pcntl
PostgreSQL	16+	Z rozszerzeniem TimescaleDB (faza 6)
Redis	7+	Cache, sesje, kolejki, pub/sub dla Reverb
Node.js	20+	Vite + React build; Puppeteer dla PDF
Composer	2.x	—

Inicjalizacja projektu Laravel

```
# Nowy projekt Laravel
composer create-project laravel/laravel clientops-api

# Kluczowe pakiety backend
composer require laravel/sanctum laravel/socialite
composer require spatie/laravel-permission spatie/laravel-activitylog
composer require spatie/laravel-media-library spatie/browsershot
composer require laravel/scout laravel/reverb laravel/horizon
composer require league/flysystem-aws-s3-v3

# Konfiguracja
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"
php artisan vendor:publish --provider="Spatie\Activitylog\ActivitylogServiceProvider"
php artisan reverb:install
php artisan horizon:install
```

Inicjalizacja projektu React (panel admina)


```
# Kluczowe pakiety frontend
npm install @tanstack/react-query react-hook-form zod
npm install react-router-dom zustand
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
npm install @tiptap/react @tiptap/starter-kit @tiptap/extension-mention
npm install echarts echarts-for-react
npm install laravel-echo pusher-js
npm install lucide-react react-select react-dropzone react-day-picker
npm install @fullcalendar/react @fullcalendar/daygrid @fullcalendar/timegrid
npm install dayjs lodash-es class-variance-authority
npm install -D tailwindcss @tailwindcss/vite
```

Konfiguracja .env (backend)


```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=clientops

REDIS_HOST=127.0.0.1
REDIS_PORT=6379

SESSION_DRIVER=redis
SESSION_DOMAIN=.clientops.pl
QUEUE_CONNECTION=redis
CACHE_STORE=redis

SANCTUM_STATEFUL_DOMAINS=app.clientops.pl,*.clientops.pl

# Google OAuth
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=https://api.clientops.pl/auth/google/callback

# Reverb WebSocket
REVERB_APP_ID=
REVERB_APP_KEY=
REVERB_APP_SECRET=
REVERB_HOST=api.clientops.pl
REVERB_PORT=8080

# MinIO / S3
FILESYSTEM_DISK=s3
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=eu-central-1
AWS_BUCKET=clientops
```

Struktura katalogów Laravel (rekomendowana)


```
    Controllers/
      Auth/        AuthController, TwoFactorController, GoogleController
      Admin/       LeadController, ClientController, TicketController...
      Client/      Portal/ — endpointy dla panelu klienta
    Middleware/
      IdentifyTenant.php
      EnsureTenantContext.php
    Requests/      FormRequest per operacja
  Models/          User, Client, Lead, Ticket...
  Policies/        LeadPolicy, ProjectPolicy, TicketPolicy...
  Services/        logika biznesowa wyciągnięta z kontrolerów
  Jobs/            SLA checks, PDF generation, SEO pipeline
  Events/          LeadStatusChangedEvent, TicketCreatedEvent...
  Listeners/       MoveKanbanCardListener, CreateSlaDeadlineListener...
  Notifications/   TwoFactorCode, TicketUpdated, SlaBreachAlert...
```

Auth, role, multi-tenancy

Fundament całego systemu. Bez tej fazy nie ma nic. Implementuj w kolejności: migracje → modele → auth → role → multi-tenancy → testy.

1. Migracje — tabela users (rozszerzona)

```
// database/migrations/xxxx_create_users_table.php
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password')->nullable();          // null dla kont OAuth
    $table->string('google_id')->nullable()->unique();
    $table->boolean('two_factor_enabled')->default(false);
    $table->string('two_factor_code')->nullable();
    $table->timestamp('two_factor_expires_at')->nullable();
    $table->timestamp('email_verified_at')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

2. AuthController — logowanie email/hasło


```
{
    public function login(LoginRequest $request): JsonResponse
    {
        if (!Auth::attempt($request->only('email', 'password'))) {
            return response()->json(['message' => 'Nieprawidłowe dane logowania'], 401);
        }

        $user = Auth::user();

        // Jeśli 2FA włączone — nie tworzymy sesji, zwracamy flagę
        if ($user->two_factor_enabled) {
            Auth::logout();
            app(TwoFactorService::class)->sendCode($user);
            return response()->json([
                'requires_2fa' => true,
                'email'        => $user->email,
            ]);
        }

        $request->session()->regenerate();
        activity('auth')->causedBy($user)->log('login');
        return response()->json(['user' => $user->load('roles')]);
    }

    public function logout(Request $request): JsonResponse
    {
        activity('auth')->causedBy(Auth::user())->log('logout');
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return response()->json(['message' => 'Wylogowano']);
    }

    public function me(): JsonResponse
    {
        return response()->json(Auth::user()->load(['roles', 'permissions']));
    }
}
```

3. GoogleController — OAuth przez Socialite


```php
{
    public function redirect(): RedirectResponse
    {
        return Socialite::driver('google')->redirect();
    }

    public function callback(Request $request): RedirectResponse
    {
        try {
            $googleUser = Socialite::driver('google')->user();
        } catch (Exception $e) {
            return redirect('https://app.clientops.pl/login?error=oauth_failed');
        }

        $user = User::updateOrCreate(
            ['google_id' => $googleUser->id],
            [
                'name'  => $googleUser->name,
                'email' => $googleUser->email,
            ]
        );

        // Domyślna rola dla nowych kont przez Google
        if ($user->wasRecentlyCreated) {
            $user->assignRole('employee');
        }

        Auth::login($user);
        activity('auth')->causedBy($user)->log('login_google');
        return redirect('https://app.clientops.pl/dashboard');
    }
}
```

4. TwoFactorController — weryfikacja kodu 2FA


```
{
    public function verify(TwoFactorRequest $request): JsonResponse
    {
        $user = User::where('email', $request->email)->firstOrFail();

        if (
            !Hash::check($request->code, $user->two_factor_code) ||
            !$user->two_factor_expires_at?->isFuture()
        ) {
            return response()->json(['message' => 'Nieprawidłowy lub wygasły kod'], 422);
        }

        $user->update([
            'two_factor_code'       => null,
            'two_factor_expires_at' => null,
        ]);

        Auth::login($user);
        $request->session()->regenerate();
        activity('auth')->causedBy($user)->log('login_2fa');
        return response()->json(['user' => $user->load('roles')]);
    }

    // Włączanie / wyłączanie 2FA z ustawień użytkownika
    public function toggle(Request $request): JsonResponse
    {
        $user = Auth::user();
        $user->update(['two_factor_enabled' => !$user->two_factor_enabled]);
        activity('settings')->causedBy($user)->log(
            $user->two_factor_enabled ? '2fa_enabled' : '2fa_disabled'
        );
        return response()->json(['two_factor_enabled' => $user->two_factor_enabled]);
    }
}
```

5. Spatie Permission — setup ról


```php
foreach ($roles as $role) {
    Role::firstOrCreate(['name' => $role, 'guard_name' => 'web']);
}

// W modelu User — sprawdzanie roli
$user->hasRole('admin');            // true/false
$user->can('update-lead', $lead);  // przez Policy

// Middleware per route group
Route::middleware(['auth:sanctum', 'role:admin|manager'])->group(function () {
    Route::apiResource('leads', LeadController::class);
});
```

6. BelongsToTenant trait

```php
// app/Traits/BelongsToTenant.php
trait BelongsToTenant
{
    protected static function booted(): void
    {
        static::addGlobalScope(new TenantScope);
        static::creating(function ($model) {
            $model->client_id ??= app('tenant')?->id;
        });
    }
}

// app/Models/Scopes/TenantScope.php
class TenantScope implements Scope
{
    public function apply(Builder $builder, Model $model): void
    {
        if ($tenant = app('tenant')) {
            $builder->where($model->getTable() . '.client_id', $tenant->id);
        }
    }
}
```

Routes dla auth

`POST /auth/login` · `POST /auth/logout` · `GET /auth/me`

`POST /auth/2fa/verify` · `POST /auth/2fa/toggle` (wymaga auth)

CRM — klienci, leady, Kanban, oferty

Kolejność implementacji

1. **Clients + Contacts** — baza CRM. Migracje, modele z BelongsToTenant, ClientPolicy.
2. **Leads + lead_activities** — enum LeadStatus, relacje, LeadPolicy.
3. **Kanban boards + cards** — polimorficzny model KanbanCard, seed kolumn.
4. **Events + Listeners** — LeadStatusChangedEvent → MoveKanbanCardListener + ActivityLog.
5. **Offers** — relacja Lead → Offer, statusy, konwersja do Client.
6. **Konwersja leadu** — LeadWonListener: tworzy Client, kopiuje kontakty, loguje.

LeadController


```
{
    public function index(Request $request): JsonResponse
    {
        $leads = Lead::with(['contact', 'assignedTo', 'kanbanCard'])
            ->filter($request->only(['status', 'assigned_to', 'search']))
            ->paginate(25);

        return response()->json($leads);
    }

    public function store(StoreLeadRequest $request): JsonResponse
    {
        $lead = Lead::create($request->validated());
        return response()->json($lead->load('contact'), 201);
    }

    public function updateStatus(Lead $lead, UpdateLeadStatusRequest $request): JsonResponse
    {
        $this->authorize('update', $lead);

        $oldStatus = $lead->status;
        $lead->update(['status' => $request->status]);

        // Event uruchamia MoveKanbanCardListener + ActivityLog listener
        event(new LeadStatusChangedEvent($lead, $oldStatus, $lead->status));

        return response()->json($lead->fresh(['kanbanCard']));
    }

    public function convert(Lead $lead): JsonResponse
    {
        $this->authorize('convert', $lead);

        if ($lead->status !== LeadStatus::Won) {
            return response()->json(['message' => 'Lead musi mieć status won'], 422);
        }

        $client = app(LeadConversionService::class)->convert($lead);
        return response()->json($client, 201);
    }
}
```

LeadConversionService


```
{
    public function convert(Lead $lead): Client
    {
        return DB::transaction(function () use ($lead) {
            $client = Client::create([
                'name'       => $lead->company_name ?? $lead->contact->full_name,
                'email'      => $lead->contact->email,
                'slug'       => Str::slug($lead->company_name ?? $lead->contact->full_name),
                'created_by' => Auth::id(),
            ]);

            // Przepisz kontakty do nowego klienta
            $lead->contacts->each(fn($c) => $c->update(['client_id' => $client->id]));

            $lead->update([
                'client_id'    => $client->id,
                'converted_at' => now(),
            ]);

            activity('crm')->causedBy(Auth::user())
                ->performedOn($lead)
                ->withProperties(['client_id' => $client->id])
                ->log('converted_to_client');

            return $client;
        });
    }
}
```

PHP Enum LeadStatus


```
{
    case New          = 'new';
    case Contacted    = 'contacted';
    case NoResponse   = 'no_response';
    case InDiscussion = 'in_discussion';
    case OfferSent    = 'offer_sent';
    case Won          = 'won';
    case Lost         = 'lost';

    public function isTerminal(): bool
    {
        return match($this) {
            self::Won, self::Lost => true,
            default               => false,
        };
    }
}
```

Tickety i SLA

TicketController

```php
// app/Http/Controllers/Admin/TicketController.php
class TicketController extends Controller
{
    public function store(StoreTicketRequest $request): JsonResponse
    {
        $policy = SlaPolicy::forClient($request->client_id)
            ->forPriority($request->priority)
            ->first();

        $ticket = Ticket::create([
            ...$request->validated(),
            'sla_policy_id'        => $policy?->id,
            'sla_response_due_at'  => $policy?->responseDeadline(),
            'sla_resolution_due_at'=> $policy?->resolutionDeadline(),
        ]);

        event(new TicketCreatedEvent($ticket));
        return response()->json($ticket->load(['client', 'project', 'slaPolicy']), 201);
    }

    public function addComment(Ticket $ticket, StoreCommentRequest $request): JsonResponse
    {
        $this->authorize('addComment', $ticket);

        $comment = $ticket->comments()->create([
            ...$request->validated(),
            'author_id'   => Auth::id(),
            'author_type' => get_class(Auth::user()),
        ]);

        // Pierwsza odpowiedź admina — zapisz czas
        if (!$ticket->first_responded_at && Auth::user() instanceof User) {
            $ticket->update(['first_responded_at' => now()]);
        }

        event(new TicketCommentedEvent($ticket, $comment));
        return response()->json($comment, 201);
    }
}
```

CheckSlaBreachesJob — job co 5 minut


```
{
    public function handle(): void
    {
        // Naruszenia response time
        Ticket::whereNull('first_responded_at')
            ->where('sla_response_due_at', '<', now())
            ->whereDoesntHave('slaIncidents', fn($q) => $q->where('breach_type', 'response'))
            ->each(function (Ticket $ticket) {
                SlaIncident::create([
                    'subject_type' => 'Ticket',
                    'subject_id'   => $ticket->id,
                    'breach_type'  => 'response',
                    'breached_at'  => $ticket->sla_response_due_at,
                    'detected_at'  => now(),
                ]);
                Notification::send(
                    User::role(['admin', 'manager'])->get(),
                    new SlaBreachAlert($ticket, 'response')
                );
            });
    }
}

// Rejestracja w routes/console.php lub Kernel.php
Schedule::job(new CheckSlaBreachesJob)->everyFiveMinutes();
```

Dokumenty i PDF

DocumentController

```
// app/Http/Controllers/Admin/DocumentController.php
class DocumentController extends Controller
{
    public function update(Document $document, UpdateDocumentRequest $request): JsonResponse
    {
        $this->authorize('update', $document);

        // Zapisz wersję przed nadpisaniem
        DocumentVersion::create([
            'document_id'  => $document->id,
            'version'      => $document->version,
            'content_json' => $document->content_json,
            'created_by'   => Auth::id(),
        ]);

        $document->update([
            'content_json' => $request->content_json,
            'content_html' => $request->content_html,
            'version'      => $document->version + 1,
        ]);

        activity('documents')->causedBy(Auth::user())->performedOn($document)->log('updated');
        return response()->json($document->fresh());
    }

    public function generatePdf(Document $document): JsonResponse
    {
        $this->authorize('view', $document);
        GenerateDocumentPdfJob::dispatch($document);
        return response()->json(['message' => 'PDF jest generowany asynchronicznie']);
    }
}
```

GenerateDocumentPdfJob


```php
{
    public function __construct(public Document $document) {}

    public function handle(): void
    {
        $html = view('pdf.document', ['document' => $this->document])->render();

        $path = Browsershot::html($html)
            ->format('A4')
            ->margins(15, 15, 15, 15)
            ->pdf();   // zwraca binarnie

        $filename = "document-{$this->document->id}-v{$this->document->version}.pdf";
        Storage::disk('s3')->put("documents/{$filename}", $path);

        $this->document->update(['pdf_path' => $filename]);

        // Powiadomienie dla autora
        Notification::send(
            $this->document->createdBy,
            new PdfReadyNotification($this->document)
        );
    }
}
```

Browsershot wymaga Node.js i Puppeteer

Na serwerze produkcyjnym: `npm install puppeteer` w katalogu dostępnym dla PHP. Ustaw `Browsershot::html(...)->setNodeBinary('/usr/bin/node')->setNpmBinary('/usr/bin/npm')` jeśli binarne nie są w PATH.

Powiadomienia i wiadomości wewnętrzne

Wszystkie zdarzenia — systemowe (SLA, ticket) i ludzkie (wiadomości, @oznaczenia) — trafiają do tabeli `notifications` jako wspólna warstwa dostarczenia. Jednocześnie wysyłany jest broadcast przez Reverb (WebSocket) dla powiadomień real-time.

Klasa notyfikacji (przykład: TicketUpdated)


```
{
    public function __construct(public Ticket $ticket) {}

    public function via(object $notifiable): array
    {
        $prefs = $notifiable->notificationPreferences
            ->firstWhere('notification_type', 'ticket_update');

        $channels = $prefs?->channels ?? ['database', 'broadcast'];

        // SMS nie w ciszy nocnej
        if (in_array('sms', $channels) && $prefs?->isQuietHours()) {
            $channels = array_diff($channels, ['sms']);
        }

        return $channels;
    }

    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject("Aktualizacja zgłoszenia: {$this->ticket->title}")
            ->line("Zgłoszenie zostało zaktualizowane.")
            ->action('Zobacz zgłoszenie', url("/tickets/{$this->ticket->id}"));
    }

    public function toArray(object $notifiable): array
    {
        return [
            'type'      => 'ticket_update',
            'ticket_id' => $this->ticket->id,
            'title'     => $this->ticket->title,
        ];
    }

    public function toBroadcast(object $notifiable): BroadcastMessage
    {
        return new BroadcastMessage($this->toArray($notifiable));
    }
}
```

ConversationController — wiadomości wewnętrzne


```
{
    public function sendMessage(Conversation $conversation, Request $request): JsonResponse
    {
        $this->authorize('participate', $conversation);

        $message = $conversation->messages()->create([
            'sender_id'   => Auth::id(),
            'sender_type' => get_class(Auth::user()),
            'body'        => $request->body,
        ]);

        // Wyślij notyfikację do wszystkich uczestników (poza nadawcą)
        $participants = $conversation->participants()
            ->where('participable_id', '≠', Auth::id())
            ->get();

        $participants->each(fn($p) => $p->participable->notify(
            new NewMessageNotification($message)
        ));

        // Wykryj @mentions i utwórz wpisy
        app(MentionService::class)->detectAndNotify($message);

        return response()->json($message->load('sender'), 201);
    }
}
```

SEO z AI — pipeline asynchroniczny

Najcięższy moduł. Dane z wielu źródeł (GSC, GA4, crawler, AI, PageSpeed) spływają asynchronicznie przez kolejki Horizon. Dopiero po zebraniu wszystkich wyników generowany jest raport audytu i powiadomienie.

Pipeline — kolejność Jobów

```php
// app/Listeners/StartAuditPipelineListener.php
class StartAuditPipelineListener
{
    public function handle(AuditRequestedEvent $event): void
    {
        $audit = $event->audit;

        // Krok 1: crawler i API zaczynają równolegle
        Bus::chain([
            new CrawlWebsiteJob($audit),
        ])->catch(function () use ($audit) {
            $audit->update(['status' => 'failed']);
        })->dispatch();

        // Google APIs równolegle z crawlem
        Bus::batch([
            new FetchGSCDataJob($audit),
            new FetchGA4DataJob($audit),
            new FetchPageSpeedJob($audit),
        ])->then(function (Batch $batch) use ($audit) {
            Bus::chain([
                new AnalyzeKeywordsJob($audit),
                new AnalyzeSemanticsJob($audit),   // AI
                new GenerateAuditReportJob($audit),
                new NotifyAuditCompleteJob($audit),
            ])->dispatch();
        })->dispatch();
    }
}
```

Konfiguracja kolejek w Horizon


```
    'production' => [
        'supervisor-default' => [
            'queue'      => ['default', 'notifications'],
            'processes'  => 5,
        ],
        'supervisor-crawlers' => [
            'queue'      => ['crawlers'],
            'processes'  => 3,
            'timeout'    => 300,
        ],
        'supervisor-ai' => [
            'queue'      => ['ai-processing', 'google-apis'],
            'processes'  => 2,
            'timeout'    => 600,
        ],
    ],
],
```

TimescaleDB dla seo_keyword_ranks

Po migracji tabeli: `SELECT create_hypertable('seo_keyword_ranks', 'time');` Partycjonuje dane time-series automatycznie. Zapytania o rankingi per słowo kluczowe per okres są 10-100x szybsze niż na zwykłej tabeli PostgreSQL przy dużych danych historycznych.

Strategia testowania

Czy warto pisać testy?

Tak — szczególnie w tym projekcie, gdzie logika biznesowa jest złożona (konwersja leadów, SLA incydenty, multi-tenancy). Bez testów każda zmiana w BelongsToTenant albo CheckSlaBreachesJob może zepsuć produkcję w sposób niewidoczny. Testy nie muszą pokrywać 100% kodu — pokryj 100% krytycznych ścieżek.

Trzy poziomy testów

Unit Tests

✓ LeadStatus::isTerminal()
✓ SlaPolicy::responseDeadline()
✓ NotificationPreferences::isQuietHours()
✓ TwoFactorService::sendCode()
✓ LeadConversionService::convert()

Feature Tests (HTTP)

✓ POST /auth/login z 2FA
✓ GET /auth/google/callback
✓ PATCH /leads/{id}/status
✓ POST /tickets (z SLA)
✓ POST /auth/2fa/toggle
✓ Tenant isolation (Cross-tenant)

Integration Tests

✓ Pełny flow konwersji leadu
✓ CheckSlaBreachesJob end-to-end
✓ Notification channels dispatch
✓ PDF generation (Browsershot)
✓ SEO pipeline (Batch jobs)

Przykład testu feature — logowanie z 2FA


```
    $user = User::factory()->create([
        'two_factor_enabled' => true,
        'password'           => Hash::make('secret'),
    ]);

    Notification::fake();

    $response = $this->postJson('/auth/login', [
        'email'    => $user->email,
        'password' => 'secret',
    ]);

    $response->assertOk()
             ->assertJson(['requires_2fa' => true]);

    $this->assertGuest(); // sesja NIE powinna być wystawiona
    Notification::assertSentTo($user, TwoFactorCodeNotification::class);
});

it('logs in after valid 2FA code', function () {
    $user = User::factory()->create([
        'two_factor_enabled'    => true,
        'two_factor_code'       => Hash::make('123456'),
        'two_factor_expires_at' => now()->addMinutes(5),
    ]);

    $this->postJson('/auth/2fa/verify', [
        'email' => $user->email,
        'code'  => '123456',
    ])->assertOk()->assertJsonPath('authenticated', true);

    $this->assertAuthenticatedAs($user);
});
```

Przykład testu — izolacja tenanta


```
    $user  = User::factory()->create()->assignRole('admin');
    $lead  = Lead::factory()->create(['client_id' => $tenant2->id]);

    // Zaloguj jako user z tenant1
    app()->instance('tenant', $tenant1);
    $this->actingAs($user);

    $this->getJson("/leads/{$lead->id}")->assertNotFound();
});
```

Jak często przeprowadzać testy?

KIEDY	JAKIE TESTY	SPOSÓB
Przed każdym commitem	Unit tests (szybkie, <5s)	git pre-commit hook: `php artisan test --filter=Unit`
Przy push / PR	Unit + Feature tests	CI (GitHub Actions): pełna suita, baza testowa
Przed deployem na prod	Wszystkie testy (+ integration)	CI pipeline — blokuje deploy jeśli testy nie przechodzą
Raz w tygodniu (cron)	Integration tests (wolne, zasoby)	Nocny job CI — pełny pipeline SEO, Browershot PDF

Rekomendowane narzędzia

Pest PHP

(składnia testów jak w przykładach powyżej) — nowocześniejszy niż PHPUnit, czytelniejszy.

GitHub Actions

— darmowe CI dla publicznych repozytoriów, konfiguracja w `.github/workflows/tests.yml`.

Paratest

— uruchamianie testów równolegle na wielu CPU, skraca czas z minut do sekund.

GitHub Actions — przykład workflow


```yaml
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: clientops_test, POSTGRES_PASSWORD: secret }
      redis:
        image: redis:7
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install --no-interaction
      - run: cp .env.testing .env
      - run: php artisan key:generate
      - run: php artisan migrate --force
      - run: php artisan test --parallel
```

DOKUMENTACJA PROJEKTU

Specyfikacja Techniczna v1.5
Architektura i Model Danych v2.0
Mapowanie Bibliotek v1.0
Struktura Frontend v1.0
Przewodnik Implementacji v1.0